Coding for Artificial Intelligence

Problem Set 6

Lab Project: The DNA Sequence Marker Finder

Name: Tanishka Hira

Date: 09/09/2026

Overview

The purpose of this assignment is to practice the use of parameterized functions with positional, keyword, and default arguments. The assignment also demonstrates Python's argument-passing behaviour using mutable lists and the `id()` function.

The program contains two main functions. The first function, `search4letters()`, searches a DNA sequence for specified marker letters. The second function, `analyze_sequence()`, demonstrates how modifying a mutable list inside a function affects the original list.

Additional experiments are included to demonstrate the effect of slicing and reassignment on list objects.

Call by Object Reference

In Python, passing an argument to a function shares a reference to the same underlying object rather than making a copy of the object.

If a mutable object, such as a list, is modified inside a function using an operation such as `append()`, the modification is visible outside the function because both variables refer to the same object.

However, if the parameter is reassigned to a new object, only the local variable is changed. The original variable outside the function remains unchanged.

Task 1: Building `search4letters`

The function `search4letters()` takes a DNA sequence and a set of target letters. The target letters have a default value of `'ATCG'`.

The function converts both strings into sets and uses set intersection to determine which target letters occur in the supplied DNA sequence.

```
def search4letters(phrase: str, letters: str = 'ATCG') -> set:
    """Return the set of target 'letters' found in a supplied '
   phrase'."""

    # set(letters) creates a unique pool of target markers
    # set(phrase) represents the genetic material to analyze
    return set(letters).intersection(set(phrase))
```

Listing 1: Task 1: search4letters function

Positional Arguments

The function is called by supplying the arguments according to their position.

```
result1 = search4letters('TGGACC', 'GC')
print("search4letters('TGGACC', 'GC'):", result1)
```

Keyword Arguments

The arguments can also be supplied using their parameter names.

```
result2 = search4letters(letters='CG', phrase='TGGACC')
print("search4letters(letters='CG', phrase='TGGACC'):", result2)
```

Default Argument

When the second argument is not supplied, the default value `'ATCG'` is used.

```
result3 = search4letters('TGGACC')
print("search4letters('TGGACC'):", result3)
```

Task 2: Tracking Memory References

The `analyze_sequence()` function demonstrates how a mutable list behaves when it is passed to a function.

The `id()` function is used to display the identity of the list before and after modification.

```python
def analyze_sequence(sequence_list, marker):
    print(
        f"Inside Function (Start) - ID: {id(sequence_list)} "
        f"| Data: {sequence_list}"
    )

    # In-place modification (mutating the shared object)
    sequence_list.append(marker)

    print(
        f"Inside Function (End) - ID: {id(sequence_list)} "
        f"| Data: {sequence_list}"
    )


dna_database = ['AATCCG', 'TGGCTA']

print(
    f"Global Scope (Before) - ID: {id(dna_database)} "
    f"| Data: {dna_database}"
)

analyze_sequence(dna_database, 'CGAT')

print(
    f"Global Scope (After) - ID: {id(dna_database)} "
    f"| Data: {dna_database}"
)
```

Listing 2: Task 2: Tracking Memory References

Discovery Challenge

Question: Why was `dna_database` modified in the global scope, even though `analyze_sequence()` never returned anything?

Answer

I think the `dna_database` was modified in the global scope because the list was passed to the function as an argument, and the function worked directly on that same list. The

parameter `sequence_list` refers to the list that was originally stored in `dna_database`. When `sequence_list.append(marker)` is executed, the function changes the existing list instead of creating a new list. Since both `sequence_list` and `dna_database` refer to the same list object, the change made inside the function can also be seen in the global scope. A `return` statement is not required because the function is modifying the original mutable list rather than returning a separate modified list. Therefore, adding `'CGAT'` inside the function should also add it to `dna_database` outside the function.

3.3 Slice Experiment

The slice experiment demonstrates the effect of passing a copy of the list using `dna_database[:]`.

A slice creates a separate list object. Therefore, when `analyze_sequence()` modifies the sliced list, the original `dna_database` remains unchanged.

```
dna_database = ['AATCCG', 'TGGCTA']

print(
    f"Global Scope (Before) - ID: {id(dna_database)} "
    f"| Data: {dna_database}"
)

analyze_sequence(dna_database[:], 'CGAT')

print(
    f"Global Scope (After) - ID: {id(dna_database)} "
    f"| Data: {dna_database}"
)
```

Listing 3: Slice Experiment

3.4 Bonus Experiment: Mutation vs. Reassignment

The following experiment demonstrates the difference between mutating a list and reassigning a local variable.

```
def analyze_sequence_rebind(sequence_list, marker):
    print(
        f"\nInside Rebind (Start) - ID: {id(sequence_list)} "
        f"| Data: {sequence_list}"
    )

```

```
    sequence_list = sequence_list + [marker]  # Rebinds local name

    print(
        f"Inside Rebind (End) - ID: {id(sequence_list)} "
        f"| Data: {sequence_list}"
    )


print("\n -- BONUS: MUTATION VS REASSIGNMENT -- ")

dna_database = ['AATCCG', 'TGGCTA']

print(
    f"Global Scope (Before) - ID: {id(dna_database)} "
    f"| Data: {dna_database}"
)

analyze_sequence_rebind(dna_database, 'CGAT')

print(
    f"Global Scope (After) - ID: {id(dna_database)} "
    f"| Data: {dna_database}"
)
```

Listing 4: Mutation vs. Reassignment

In this case, the expression

```
sequence_list = sequence_list + [marker]
```

creates a new list. The local parameter is then reassigned to this new list, while the original `dna_database` remains unchanged.

Edge Cases

Several edge cases were tested to verify that the program behaves correctly under unusual or boundary inputs.

Edge Case 1: Empty DNA Sequence

```
empty_result = search4letters('')

print("search4letters(''):", empty_result)
```

The expected result is an empty set because there are no characters in the supplied phrase.

Edge Case 2: Positional and Keyword Arguments

```
mixed_result = search4letters('TGGACC', letters='GC')

print(
    "search4letters('TGGACC', letters='GC'):",
    mixed_result
)
```

This confirms that a positional argument can be followed by a keyword argument.

Edge Case 3: Repeated Slice Calls

```
dna_database = ['AATCCG', 'TGGCTA']

print("Original database:", dna_database)

analyze_sequence(dna_database[:], 'CGAT')
analyze_sequence(dna_database[:], 'AAAA')

print(
    "\nDatabase after repeated slice calls:",
    dna_database
)
```

Each use of `dna_database[:]` creates a new list. Therefore, the original database remains unchanged after both function calls.

Conclusion

The program successfully demonstrates the use of positional, keyword, and default function arguments through the `search4letters()` function. It also demonstrates Python's call-by-object-reference behaviour using a mutable list.

The `id()` values show that the function and global variable refer to the same list during in-place mutation. In contrast, slicing creates a separate list, while reassignment creates a new object for the local variable.

The edge cases further verify that the functions behave as expected for empty input, mixed argument styles, and repeated slice operations.